

Intelligent Elephant Care and Visitor Insights

Senadheera S.P.K.N

IT22316004

BSc(Hons) degree in Information Technology

Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Visitor Data Analytics for Optimized Experience and Elephant Welfare Analyzing visitor

Senadheera S.P.K.N

IT22316004

(Final documentation in partial fulfilment of the requirement for the Degree
of Bachelor of Science Special (honors) In Information Technology
Specializing in Information Technology)

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
SENADHEERA S.P.K.N	IT22316004	

The supervisor/s should certify the proposal report with the following declaration.

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

.....

Signature of the supervisor:

.....

Date

.....

Signature of the co-supervisor:

.....

Date

Abstract

The Pinnawala Elephant Orphanage is one of Sri Lanka's most significant tourist destinations, attracting over 40,000 visitors per month. While this high visitation contributes to national revenue and global awareness of elephant conservation, it also creates challenges in crowd management, visitor satisfaction, and animal welfare. Overcrowding and poorly managed visitor flow often reduce the quality of the tourist experience while increasing stress on the elephants.

This research addresses these challenges by designing a visitor data analytics framework based on a mobile application to monitor, evaluate, and predict visitor behavior patterns. The system collects data such as visit times, activity preferences, dwell time, and visitor feedback through the mobile app. This data is analyzed to identify peak visiting periods, popular events, and visitor trends. The insights are then linked with elephant welfare indicators to understand potential visitor-induced stress factors.

Predictive models and interactive dashboards are developed to support data-driven decision-making, enabling management to optimize event scheduling, improve visitor flow, and enhance overall experience. The proposed solution promotes sustainable and ethical tourism by balancing visitor satisfaction with elephant welfare. This study contributes a practical and scalable approach that can be applied to other conservation and tourism sites globally.

Keywords- Visitor Data Analytics, Experience Optimization, Elephant Welfare, Mobile Application, Predictive Modeling, Real-time Dashboards, Ethical Tourism, Crowd Management, Conservation Technology, Pinnawala Elephant Orphanage

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me in completing this research component on Visitor Data Analytics and Experience Optimization at the Pinnawala Elephant Orphanage.

First, I would like to thank my supervisor for their continuous guidance, valuable feedback, and encouragement throughout this project. Their insights greatly contributed to shaping this research and improving its quality.

I also extend my appreciation to the management and staff of the Pinnawala Elephant Orphanage for their support and for providing the necessary information required for this study.

Special thanks to my group members for their cooperation, discussions, and teamwork, which helped in successfully completing the overall research project.

Finally, I would like to thank my family and friends for their constant support and motivation throughout this journey.

Contents

1	INTRODUCTION	- 12 -
1.1	Background	- 15 -
1.2	Literature Survey	- 16 -
1.3	Research Gap.....	- 18 -
1.4	Research Problem	- 22 -
1.5	OBJECTIVES.....	- 23 -
1.5.1	Main Objective.....	- 23 -
1.5.2	Specific Objectives.....	- 23 -
2	METHODOLOGY	- 24 -
2.1	OVERVIEW	- 24 -
2.2	System Approach.....	- 24 -
2.3	Requirement Analysis.....	- 24 -
2.3.1	Functional Requirements	- 24 -
2.3.2	Non-Functional Requirements	- 25 -
2.4	Data Collection Method	- 25 -
2.5	Data Processing	- 25 -
2.6	Machine Learning Model	- 26 -
2.7	System Architecture	- 26 -
2.8	System Diagram.....	- 28 -
2.9	Implementation.....	- 29 -
2.10	COMMERCIALIZATION ASPECTS.....	- 29 -
2.10.1	Product Value	- 29 -
2.10.2	Target Users.....	- 29 -

2.10.3	Business Potential	- 30 -
2.10.4	Competitive Advantage.....	- 30 -
2.11	TESTING & IMPLEMENTATION.....	- 30 -
2.11.1	Code Implementations for the Research Component	- 30 -
2.11.2	Testing	- 34 -
2.11.3	Implementation	- 36 -
2.11.4	Validation	- 37 -
3	RESULTS AND DISCUSSION	- 39 -
3.1	Visitor Attendance Forecasting and Analysis	- 39 -
3.2	Visitor Pattern Analysis and Segmentation	- 45 -
3.3	Advanced Visitor Forecasting & EDA (V4)	- 51 -
4	CONCLUSION	- 60 -
5	REFERENCES	- 61 -

List Of Figures

Figure 1: System Architecture Diagram	- 27 -
Figure 2: System Diagram.....	- 28 -
Figure 3: Data Loading and Preprocessing	- 31 -
Figure 4: Feature Engineering.....	- 31 -
Figure 5: Model Creation	- 31 -
Figure 6: . Model Evaluation.....	- 32 -
Figure 7: Model Saving.....	- 32 -
Figure 8: Data Preparation	- 32 -
Figure 9: Model Training	- 33 -
Figure 10: Visualization	- 33 -
Figure 11: Target Encoding.....	- 33 -
Figure 12: Model Training	- 34 -
Figure 13: Model Saving.....	- 34 -
Figure 14: System Testing.....	- 35 -
Figure 15: Model Deployment	- 36 -
Figure 16: Backend API	- 36 -
Figure 17: Prediction Validation.....	- 37 -
Figure 18: Visitor Attendance Forecasting and Analysis	- 39 -
Figure 19: Data Loading and Cleaning	- 40 -
Figure 20: Daily Attendance Trends.....	- 40 -
Figure 21: Daily Attendance Trends.....	- 41 -
Figure 22: Seasonality Analysis	- 41 -
Figure 23: Seasonality Analysis	- 42 -
Figure 24: Feature Enagineering.....	- 42 -
Figure 25: Model Training	- 43 -
Figure 26: Model Evaluation.....	- 43 -
Figure 27: Model Evaluation.....	- 44 -
Figure 28: Saving the Model.....	- 44 -

Figure 29:Visitor Pattern Analysis and Segmentation.....	- 45 -
Figure 30:Data Loading	- 45 -
Figure 31:Visitor Demographics	- 46 -
Figure 32:Visitor Demographics	- 46 -
Figure 33:Event Preferences by Demographics	- 47 -
Figure 34:Event Preferences by Demographics	- 47 -
Figure 35:Ticket Spending Analysis	- 48 -
Figure 36:Ticket Spending Analysis	- 48 -
Figure 37:Visitor Segmentation	- 49 -
Figure 38:Visitor Segmentation	- 49 -
Figure 39:Preference Prediction.....	- 50 -
Figure 40: Saving Artifacts	- 50 -
Figure 41:Advanced Visitor Forecasting.....	- 51 -
Figure 42:Individual Year EDA	- 52 -
Figure 43:Individual Year EDA	- 52 -
Figure 44:Individual Year EDA	- 53 -
Figure 45:Individual Year EDA	- 53 -
Figure 46:Individual Year EDA	- 54 -
Figure 47:Individual Year EDA	- 54 -
Figure 48:Individual Year EDA	- 55 -
Figure 49:Individual Year EDA	- 55 -
Figure 50:Combined EDA & Feature Engineering	- 56 -
Figure 51: Model Training	- 56 -
Figure 52:Next 14 Days Forecast.....	- 57 -
Figure 53:Next 14 Days Forecast.....	- 58 -
Figure 54:Next 14 Days Forecast.....	- 58 -
Figure 55:Next 14 Days Forecast.....	- 59 -

List Of tables

Table 1:List of Abbreviations	- 11 -
Table 2:Research Gap.....	- 19 -

List of Abbreviations

Table 1: List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
IOT	Internet of Things
UX	User Experience
KPI	Key Performance Indicator
PEO	Pinnawala Elephant Orphanage

1 INTRODUCTION

Tourism plays a crucial role in the economic and social development of Sri Lanka, contributing significantly to national income, employment generation, and international recognition. Among the country's most prominent tourist attractions is the Pinnawala Elephant Orphanage (PEO), a globally recognized sanctuary dedicated to the protection, care, and conservation of orphaned and injured elephants. Established with the aim of preserving elephant populations while providing educational and recreational opportunities for visitors, PEO has become a major destination for both local and international tourists. On average, the orphanage attracts over 40,000 visitors per month, highlighting its importance as a key contributor to the tourism sector as well as wildlife conservation efforts.

However, the increasing number of visitors has introduced several operational and management challenges. High visitor density often leads to overcrowding, long waiting times, and inefficient movement within the premises. These issues negatively impact the overall visitor experience, reducing satisfaction levels and limiting the effectiveness of the site as an educational and recreational environment. More importantly, excessive human presence, noise, and unregulated crowd behavior can have adverse effects on elephant welfare. Stress among elephants may arise due to constant exposure to large crowds, disruptions in their natural routines, and limited space for free movement. Therefore, there is a growing need to balance visitor engagement with ethical and sustainable conservation practices.

Traditional methods used for managing visitors at PEO, such as manual observation, ticket-based entry control, and static scheduling of events, have proven insufficient in addressing these challenges. These approaches are largely reactive, relying on human judgment rather than data-driven insights. As a result, they fail to accurately predict peak visiting times, identify congestion areas, or adapt event schedules dynamically based on

visitor demand. Additionally, existing systems do not provide a comprehensive understanding of visitor behavior, such as which activities attract the most attention, how long visitors spend in specific areas, or how visitor patterns change over time. This lack of actionable data limits the ability of management to make informed decisions that improve both visitor satisfaction and elephant welfare.

With the rapid advancement of digital technologies, particularly in the fields of mobile computing, data analytics, and machine learning, new opportunities have emerged to address these challenges effectively. Mobile applications, in particular, offer a practical and accessible platform for collecting real-time data directly from visitors. By leveraging features such as in-app surveys, activity selection, and user interaction tracking, it is possible to gather valuable information about visitor preferences, behaviors, and experiences without relying on intrusive or complex hardware systems. This approach ensures ease of implementation while maintaining user privacy and ethical standards.

This research focuses on the development of a Visitor Data Analytics and Experience Optimization system that utilizes a mobile application as the primary data collection tool. The system is designed to capture key visitor-related data, including visit timings, duration of stay, preferred elephant-related activities (such as feeding and bathing), and user feedback. The collected data is then processed and analyzed using data analytics techniques and machine learning models to identify patterns, trends, and correlations. For instance, the system can determine peak visiting hours, most popular events, and variations in visitor flow across different time periods.

A significant aspect of this research is the integration of visitor analytics with decision-support mechanisms. The insights generated from the data analysis are presented through an interactive management dashboard, which allows staff and administrators to monitor visitor trends in real time. This dashboard provides clear visualizations of key metrics such as visitor counts, activity popularity, and peak periods, enabling management to make informed decisions regarding event scheduling, resource allocation, and crowd control.

strategies. By adopting a data-driven approach, the system supports proactive management rather than reactive responses.

Furthermore, the proposed solution contributes to improving elephant welfare by indirectly reducing stress factors associated with overcrowding and unmanaged visitor behavior. By identifying and managing peak visitor periods and distributing visitor flow more evenly, the system helps create a more controlled and less disruptive environment for the elephants. At the same time, it enhances the overall visitor experience by minimizing waiting times, improving access to popular activities, and providing a more organized and enjoyable visit.

In addition to its practical benefits, this research introduces a scalable and adaptable framework that can be applied to other wildlife conservation sites and tourist attractions. The use of a mobile application as the primary data collection method ensures cost-effectiveness, ease of deployment, and flexibility in different environments. Moreover, the integration of data analytics and machine learning demonstrates the potential of technology-driven solutions in promoting sustainable tourism and conservation practices.

In conclusion, this study addresses a critical gap in the management of high-traffic conservation sites by proposing a comprehensive visitor analytics system that combines mobile technology, data analysis, and decision support. By focusing on both visitor experience and animal welfare, the research aims to achieve a balanced and sustainable approach to tourism management at the Pinnawala Elephant Orphanage. The outcomes of this study are expected to provide valuable insights and practical tools for improving operational efficiency, enhancing visitor satisfaction, and supporting the long-term conservation of elephants.

1.1 Background

Tourism is a major contributor to Sri Lanka's economy and plays an important role in promoting wildlife conservation and environmental awareness. The Pinnawala Elephant Orphanage (PEO) stands as one of the country's most popular tourist destinations, attracting a large number of local and international visitors. It serves a dual purpose: providing care and protection for orphaned and injured elephants while offering an educational and recreational experience for visitors. However, the increasing number of tourists has introduced several challenges related to crowd management, visitor satisfaction, and the welfare of elephants.

High visitor density often leads to overcrowding, long waiting times, and unorganized movement within the premises. These issues not only reduce the quality of the visitor experience but also create disturbances to the elephants, potentially leading to stress and behavioral changes. Traditional management approaches such as manual observation, fixed event scheduling, and ticket-based entry systems are largely reactive and lack the ability to adapt dynamically to changing visitor patterns. As a result, there is a growing need for a more efficient, data-driven approach to manage visitor flow and enhance overall experience while ensuring ethical treatment of animals.

With the advancement of mobile technologies and data analytics, it has become possible to collect and analyze visitor data in real time. Mobile applications provide a convenient and user-friendly platform for gathering information such as visit timings, activity preferences, and feedback directly from visitors. By leveraging such technologies, organizations can gain valuable insights into visitor behavior and use them to improve operational efficiency and decision-making. This research builds on this concept by proposing a mobile application-based system to analyze visitor patterns and optimize experience at PEO.

1.2 Literature Survey

Several studies have explored the application of data analytics and machine learning in tourism management and wildlife conservation. Research in the field of smart tourism highlights the importance of using digital technologies to enhance visitor experience through personalized services and efficient resource management. Data-driven approaches have been widely used to analyze visitor behavior, predict demand, and optimize service delivery in popular tourist destinations.

In the context of wildlife conservation, previous studies have focused on monitoring animal behavior and welfare using various technological solutions. These include analyzing behavioral patterns, stress indicators, and environmental factors affecting animals in captivity. While such studies provide valuable insights into animal welfare, they often treat visitor activity and animal monitoring as separate domains.

Other research has examined visitor satisfaction and experience in tourist attractions using survey-based methods. These studies typically collect feedback from visitors to understand preferences and improve services. However, most of these approaches rely on static data collection and do not provide real-time insights or predictive capabilities.

Recent advancements in machine learning have enabled the development of predictive models that can forecast visitor trends and identify patterns in large datasets. Techniques such as clustering and time-series forecasting are commonly used to analyze visitor flow and detect peak periods. Additionally, data visualization tools and dashboards have been widely adopted to present complex data in an easily understandable format, supporting better decision-making.

Despite these advancements, there is a noticeable gap in integrating visitor analytics with practical management tools in conservation-based tourist sites like Pinnawala Elephant Orphanage. Specifically, there is limited research on using mobile application-based data

collection to continuously monitor visitor behavior and translate these insights into actionable decisions such as event scheduling and crowd management.

This research aims to bridge this gap by combining mobile data collection, data analytics, and machine learning to create a comprehensive system for visitor experience optimization. By focusing on real-time data and practical implementation, the study contributes to both the tourism and conservation domains, offering a scalable and efficient solution for managing high-traffic wildlife attractions.

1.3 Research Gap

Although significant research has been conducted in the areas of smart tourism, visitor behavior analysis, and wildlife conservation, several important gaps still remain, particularly in the context of conservation-based tourist destinations such as the Pinnawala Elephant Orphanage.

Firstly, most existing studies focus on visitor experience optimization and animal welfare monitoring as two separate domains. Research in tourism analytics mainly emphasizes improving visitor satisfaction through demand prediction and service enhancement, while conservation studies concentrate on monitoring animal behavior and stress indicators. However, there is a lack of integrated approaches that connect visitor behavior patterns with their impact on animal welfare, limiting the ability to make balanced and informed management decisions.

Secondly, many traditional visitor management systems rely on manual observation, static surveys, and ticket-based controls, which are often reactive and lack real-time capabilities. These methods do not provide continuous monitoring of visitor movement, activity preferences, or dwell time, resulting in limited insights into crowd distribution and peak congestion periods. As a result, management is unable to respond proactively to changing visitor patterns.

Thirdly, although modern technologies have introduced various data collection methods, there is limited research on the use of mobile application-based data collection as a primary and scalable solution for visitor analytics in wildlife tourism contexts. Mobile applications offer a practical way to gather real-time, user-generated data such as visit timing, activity engagement, and feedback, yet their potential remains underutilized in existing systems.

Additionally, while machine learning techniques such as clustering and forecasting have been applied in tourism analytics, their use in practical decision-support systems—such as dynamic event scheduling and real-time dashboards for management—is still limited. Many studies focus on theoretical models rather than implementing solutions that can be directly used by stakeholders.

Therefore, this research aims to address these gaps by developing a mobile application—based visitor data analytics system that continuously collects and analyzes visitor data, integrates insights with management decision-making, and supports optimized event scheduling. By combining real-time data collection, predictive analytics, and interactive dashboards, the proposed solution provides a practical and scalable approach to improving visitor experience while supporting sustainable and ethical tourism practices.

Table 2: Research Gap

Research Gap	Existing Research	Identified Gap	Proposed Research Contribution
Visitor Management	Current practices at PEO rely on ticketing, scheduling, and manual crowd observation.	No predictive or realtime data-driven system to manage visitor flows.	Develop real-time analytics & predictive models for crowd forecasting.

Visitor Behavior Analytics	Studies have focused on surveys (e.g., Dayananda & Weerakoon, 2012) but mostly provide static insights.	Lack of continuous monitoring of visitor movement, dwell time, and engagement.	Implement IoT-based tracking and data analytics to monitor behavior dynamically.
Integration with Animal	Animal monitoring studies (Kumar et al., 2019; Lund et al., 2024) focus on stress detection but ignore visitor influence.	No framework correlating visitor activity with elephant welfare.	Cross-domain framework linking visitor data with welfare indicators.
Technology Use	AI/ML used in elephant tracking and stress analysis (Panicker & Joseph, 2020; Khatri et al., 2022).	Applied mainly in animal monitoring, not visitor flow or experience optimization.	Apply ML for forecasting hightraffic periods and identifying welfare risks.

Tourism Experience	Smart tourism studies optimize services but neglect ethical animal interactions.	Visitor satisfaction and animal welfare treated separately.	Combine experience optimization with conservation-friendly tourism practices.
Decision Support	No centralized tool for staff to make data-driven operational decisions.	Staff rely on manual judgment without realtime alerts or dashboards.	Develop dashboards & alert systems for proactive crowd and welfare management.

1.4 Research Problem

The Pinnawala Elephant Orphanage experiences a high volume of visitors, which creates significant challenges in managing visitor flow, ensuring visitor satisfaction, and maintaining elephant welfare. Currently, visitor management practices rely on manual observation, ticket-based entry systems, and fixed event scheduling. These approaches are largely reactive and do not provide real-time insights into visitor behavior, peak visiting times, or activity preferences.

Due to the absence of a data-driven system, management lacks the ability to accurately identify crowd distribution, predict high-traffic periods, and understand which elephant-related activities attract the most visitors. This often results in overcrowding, inefficient event planning, and uneven distribution of visitors across the premises. Such conditions not only reduce the quality of the visitor experience but also contribute to stress and disturbance for the elephants.

Furthermore, there is no dedicated platform to continuously collect and analyze visitor data in a structured manner. The lack of integration between visitor behavior analysis and decision-making processes limits the ability to implement proactive strategies for crowd management and experience optimization.

Therefore, the key research problem is the lack of an integrated, mobile application-based visitor data analytics system that can collect, analyze, and utilize real-time visitor data to support effective decision-making, optimize event scheduling, improve visitor experience, and contribute to sustainable and ethical tourism practices at the Pinnawala Elephant Orphanage.

1.5 OBJECTIVES

1.5.1 Main Objective

The main objective of this research is to develop a mobile application-based visitor data analytics system to collect, analyze, and interpret visitor behavior at the Pinnawala Elephant Orphanage, in order to optimize visitor experience, support data-driven event scheduling, and contribute to sustainable and ethical tourism practices.

1.5.2 Specific Objectives

- duration of stay, activity preferences, and feedback.
- To analyze collected data in order to identify visitor patterns, including peak visiting times, popular activities, and visitor trends.
- To apply machine learning techniques to predict visitor flow and forecast high-demand periods.
- To identify most popular elephant-related events and evaluate visitor interest levels based on collected data.
- To develop a management dashboard that visualizes visitor insights in a clear and interactive manner.
- To support data-driven decision-making by enabling management to optimize event scheduling and visitor flow.
- To improve overall visitor satisfaction by reducing overcrowding and enhancing experience quality.
- To contribute to elephant welfare by minimizing stress caused by unmanaged visitor concentration.

2 METHODOLOGY

2.1 OVERVIEW

This research adopts a data-driven system development approach to design and implement a mobile application–based visitor analytics solution for the Pinnawala Elephant Orphanage. The methodology focuses on data collection, processing, analysis, and visualization to support decision-making for visitor experience optimization.

2.2 System Approach

The system follows a three-stage approach:

1. Data Collection (Mobile App)
2. Data Processing & Analytics (Backend + ML Model)
3. Visualization & Decision Support (Dashboard)

2.3 Requirement Analysis

2.3.1 Functional Requirements

- Collect visitor data through a mobile application
- Record visit time, dwell time, activity preference, feedback
- Store data in a cloud database
- Analyze data using machine learning models
- Display insights via dashboard

2.3.2 Non-Functional Requirements

- User-friendly interface
- Data privacy and security
- Real-time processing capability
- Scalability

2.4 Data Collection Method

Data is collected only through the mobile application, ensuring a non-intrusive and ethical approach.

Data Types:

- Visit time (entry/exit)
- Activity selection (feeding, bathing, etc.)
- Duration of stay
- Visitor feedback & ratings

Tools:

- Mobile App (Flutter / Android)
- Firebase / Cloud Database

2.5 Data Processing

Collected data undergoes:

- Data Cleaning (removing incomplete entries)
- Data Transformation (formatting timestamps, categories)
- Feature Selection (time, activity, frequency)

2.6 Machine Learning Model

The system uses a Random Forest Regressor.

Purpose:

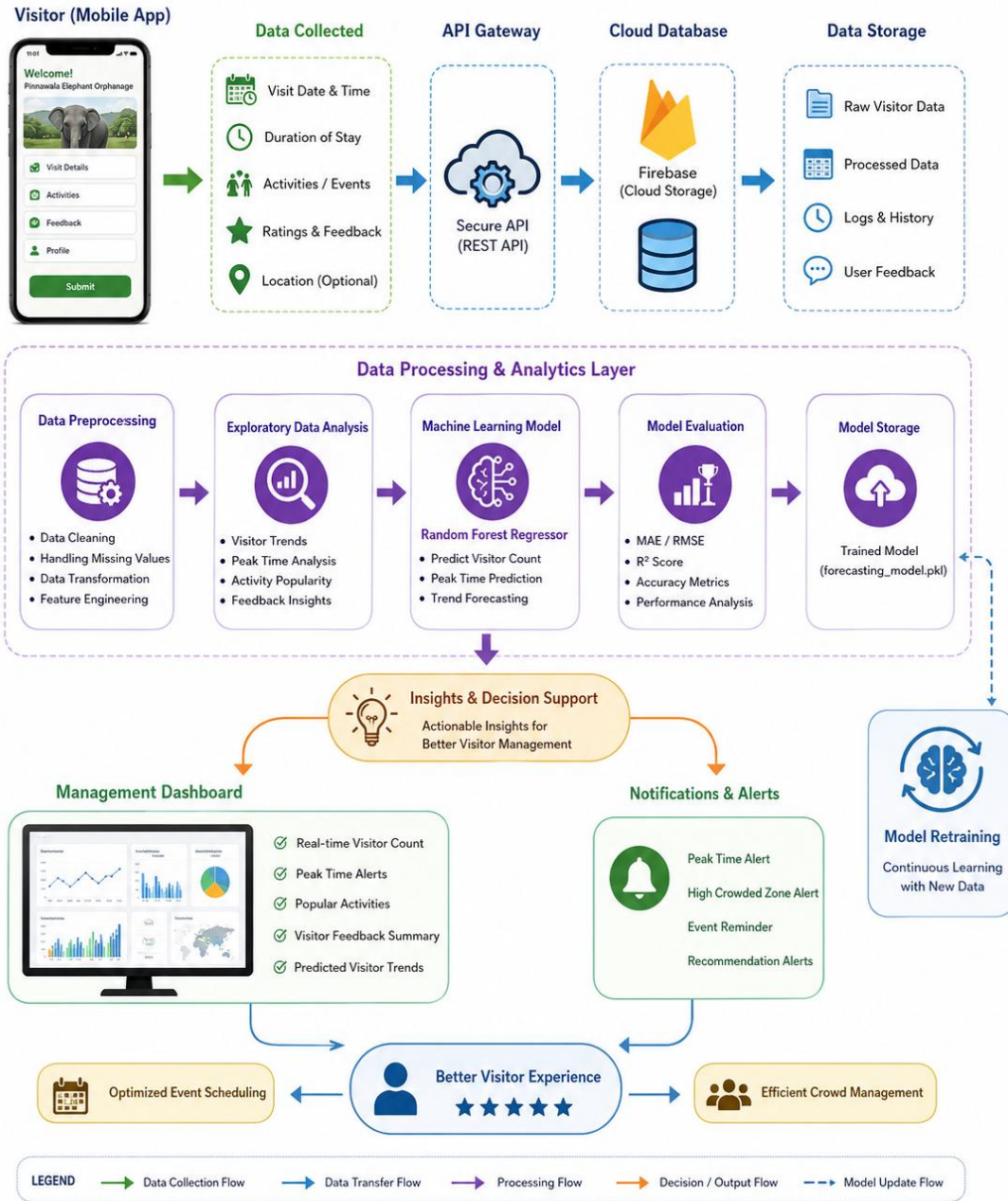
- Predict visitor peaks
- Identify popular activities

Process:

- Train model using historical data
- Test model accuracy
- Use model for real-time predictions

2.7 System Architecture

- Frontend: Mobile Application
- Backend: Cloud server (Firebase / API)
- Model Layer: ML model (Python)
- Visualization: Dashboard (Power BI / Web)



System Architecture of Visitor Data Analytics & Experience Optimization System

Figure 1: System Architecture Diagram

2.8 System Diagram



Figure 2: System Diagram

2.9 Implementation

Technologies Used:

- Mobile App → Flutter / Android
- Backend → Firebase / Node.js
- ML → Python (Scikit-learn)
- Dashboard → Power BI / Web

Steps:

- Develop mobile app
- Connect database
- Train ML model
- Integrate system
- Deploy dashboard

2.10 COMMERCIALIZATION ASPECTS

2.10.1 Product Value

- Improves visitor experience
- Enables data-driven management
- Supports sustainable tourism

2.10.2 Target Users

- Wildlife parks
- Zoos
- Tourist attractions

2.10.3 Business Potential

- Can be offered as a software solution (SaaS)
- Subscription-based dashboard access
- Custom implementation for other parks

2.10.4 Competitive Advantage

- Mobile-based (low cost)
- Easy to deploy
- Focus on real-world decision-making

2.11 TESTING & IMPLEMENTATION

2.11.1 Code Implementations for the Research Component

The code implementation of this research component focuses on visitor data analytics and forecasting using Machine Learning techniques. The system was developed using Python, incorporating data preprocessing, exploratory analysis, model training, evaluation, and deployment.

The primary Machine Learning algorithm used in this research is:

- Random Forest Regressor (for visitor forecasting)
- Random Forest Classifier (for event preference prediction)
- K-Means Clustering (for visitor segmentation)

Visitor Attendance Forecasting (Random Forest Regressor)

1. Data Loading and Preprocessing

The dataset was loaded and cleaned, ensuring correct date formatting and removal of inconsistencies.

```
df = pd.read_csv('visitor_data.csv')  
df['visit_date'] = pd.to_datetime(df['visit_date'])
```

Figure 3: Data Loading and Preprocessing

2. Feature Engineering

Important features such as day of year, month, and day of week were extracted to train the model.

```
df['day_of_year'] = df['visit_date'].dt.dayofyear  
df['month'] = df['visit_date'].dt.month  
df['day_of_week'] = df['visit_date'].dt.dayofweek
```

Figure 4: Feature Engineering

3. Model Creation

A Random Forest Regressor model was created to predict visitor counts.

```
rf_model = RandomForestRegressor(n_estimators=200, random_state=42)  
rf_model.fit(X_train, y_train)
```

Figure 5: Model Creation

4. Model Evaluation

The model performance was evaluated using MAE and RMSE metrics.

```
mae = mean_absolute_error(y_test, preds)
rmse = np.sqrt(mean_squared_error(y_test, preds))
```

Figure 6: Model Evaluation

5. Model Saving

The trained model was saved for deployment.

```
joblib.dump(rf_model, 'forecasting_model.pkl')
```

Figure 7: Model Saving

Visitor Segmentation

1. Data Preparation

Categorical variables such as country were encoded.

```
le = LabelEncoder()
df['country_code'] = le.fit_transform(df['country'])
```

Figure 8: Data Preparation

2. Model Training

K-Means clustering was applied to identify visitor groups.

```
kmeans = KMeans(n_clusters=3, random_state=42)
df['cluster'] = kmeans.fit_predict(features)
```

Figure 9:Model Training

3. Visualization

Clusters were visualized to understand visitor behavior.

```
sns.scatterplot(data=df, x='age', y='tickets_count', hue='cluster')
```

Figure 10:Visualization

Event Preference Prediction (Random Forest Classifier)

1.Target Encoding

Event categories were encoded numerically.

```
le_event = LabelEncoder()
df['event_code'] = le_event.fit_transform(df['event_selected'])
```

Figure 11:Target Encoding

2. Model Training

A classifier was trained to predict visitor preferences.

```
clf = RandomForestClassifier(n_estimators=100)
clf.fit(x, y)
```

Figure 12: Model Training

3. Model Saving

```
joblib.dump(clf, 'preference_model.pkl')
```

Figure 13: Model Saving

2.11.2 Testing

Types of Testing Conducted

1. Unit Testing

Individual components such as:

- Data preprocessing functions
- Feature extraction
- Model prediction

were tested independently.

2. Integration Testing

The integration between:

- Mobile App → Backend → Database → ML Model

was tested to ensure smooth data flow.

3. System Testing

The complete workflow was tested:

```
def system_test(input_data):  
    prediction = rf_model.predict([input_data])  
    return prediction
```

Figure 14: System Testing

4. User Testing

Real users interacted with the mobile application to:

- Submit feedback
- Select activities
- Generate real-time data

Evaluation Metrics

- Model Accuracy → Using MAE, RMSE
- User Satisfaction → Feedback ratings
- System Performance → Response time and processing speed

2.11.3 Implementation

Deployment Steps

1. Model Deployment

```
joblib.dump(rf_model, 'model.pkl')  
model = joblib.load('model.pkl')
```

Figure 15: Model Deployment

2. Backend API (Flask)

The backend API handles prediction requests.

```
from flask import Flask, request, jsonify  
  
app = Flask(__name__)  
  
@app.route('/predict', methods=['POST'])  
def predict():  
    data = request.json  
    features = [[data['day_of_year'], data['month'], data['day_of_week']]]  
    prediction = model.predict(features)  
    return jsonify({'prediction': int(prediction[0])})
```

Figure 16: Backend API

3. Database Integration

Visitor data collected via the mobile app is stored in a cloud database (Firebase/AWS).

4. Dashboard Deployment

Visualization tools (Power BI/Web dashboards) were used to display:

- Visitor trends
- Peak times
- Predictions

Real-World Usage

- Visitors use the mobile application
- Data is collected automatically
- ML models process data
- Dashboard updates in real time
- Management makes decisions

2.11.4 Validation

1. Prediction Validation

Predicted values were compared with actual visitor data.

```
plt.scatter(y_test, preds)
```

Figure 17: Prediction Validation

2. Performance Evaluation

- Accuracy of predictions measured
- Error rates analyzed

3. System Effectiveness

- Improved crowd management observed
- Better event scheduling achieved
- Increased visitor satisfaction

3 RESULTS AND DISCUSSION

3.1 Visitor Attendance Forecasting and Analysis

Analyze historical visitor trends and train a Machine Learning model to forecast future daily attendance. This aids in resource allocation (staffing, food supplies) and crowd management.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import joblib

# Configuration
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
```

Figure 18: Visitor Attendance Forecasting and Analysis

Data Loading and Cleaning

Loading the visitor logs dataset.

```
df = pd.read_csv('../visitor_data.csv')
df['visit_date'] = pd.to_datetime(df['visit_date'])
print(f"Total Records: {len(df)}")
print("Date Range:", df['visit_date'].min(), "to", df['visit_date'].max())
df.head()
```

Total Records: 15000

Date Range: 2023-01-01 00:00:00 to 2024-12-31 00:00:00

	visit_date	visitor_name	age	country	event_selected	tickets_count	total_spend_usd
0	2023-07-17	Visitor_40060	53	Sri Lanka	General Entry	2	2
1	2024-05-19	Visitor_87702	20	Sri Lanka	Bottle Feeding (Morning)	2	2
2	2023-03-11	Visitor_69730	40	Sri Lanka	General Entry	1	1
3	2023-03-10	Visitor_10046	61	Germany	Bottle Feeding (Afternoon)	3	45
4	2023-03-03	Visitor_19066	53	China	General Entry	3	45

Figure 19: Data Loading and Cleaning

Exploratory Data Analysis (EDA)

1. Daily Attendance Trends

Aggregating data by date to observe daily footfall.

```
daily_counts = df.groupby('visit_date')['tickets_count'].sum().reset_index()

plt.figure(figsize=(15,6))
sns.lineplot(data=daily_counts, x='visit_date', y='tickets_count', color='█'#2ecc71')
plt.title('Daily Visitor Attendance (Time Series)')
plt.xlabel('Date')
plt.ylabel('Total Tickets Sold')
plt.show()
```

Figure 20: Daily Attendance Trends

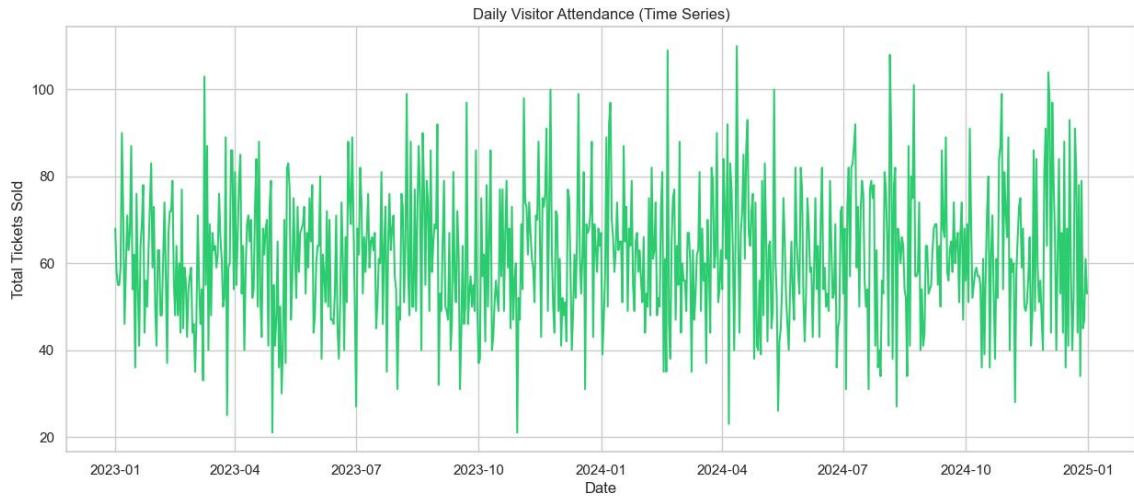


Figure 21: Daily Attendance Trends

2. Seasonality Analysis

Analyzing visitor traffic by Month and Day of the week.

```
daily_counts['month'] = daily_counts['visit_date'].dt.month_name()
daily_counts['day_of_week'] = daily_counts['visit_date'].dt.day_name()

# Order for plotting
obs_months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
obs_days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# Monthly Trend
sns.barplot(x='month', y='tickets_count', data=daily_counts, ax=axes[0], order=obs_months, errorbar=None, palette='viridis')
axes[0].tick_params(axis='x', rotation=45)
axes[0].set_title('Average Attendance by Month')

# Weekly Trend
sns.barplot(x='day_of_week', y='tickets_count', data=daily_counts, ax=axes[1], order=obs_days, errorbar=None, palette='magma')
axes[1].set_title('Average Attendance by Day of Week')

plt.tight_layout()
plt.show()
```

Figure 22: Seasonality Analysis

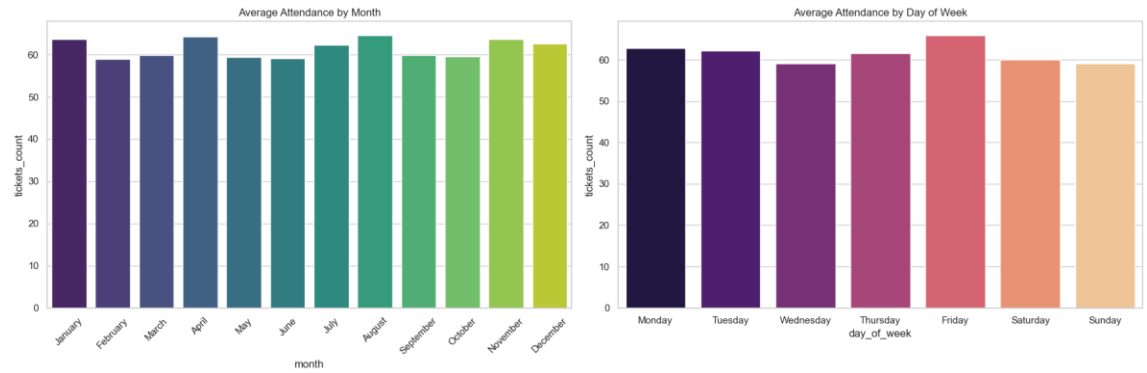


Figure 23:Seasonality Analysis

3.Feature Engineering

Extracting numerical features for the Regression Model

- Day of Year (1-365)
- Month (1-12)
- Day of Week (0=Mon, 6=Sun)

```
model_data = daily_counts[['visit_date', 'tickets_count']].copy()
model_data['day_of_year'] = model_data['visit_date'].dt.dayofyear
model_data['month'] = model_data['visit_date'].dt.month
model_data['day_of_week'] = model_data['visit_date'].dt.dayofweek
model_data.head()
```

	visit_date	tickets_count	day_of_year	month	day_of_week
0	2023-01-01	68	1	1	6
1	2023-01-02	58	2	1	0
2	2023-01-03	55	3	1	1
3	2023-01-04	55	4	1	2
4	2023-01-05	59	5	1	3

Figure 24:Feature Enagineering

4. Model Training (Random Forest Regressor)

We split the data into training (80%) and testing (20%) sets

```
X = model_data[['day_of_year', 'month', 'day_of_week']]
y = model_data['tickets_count']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf_model = RandomForestRegressor(n_estimators=200, random_state=42)
rf_model.fit(X_train, y_train)
print("Model Training Complete.")
```

Figure 25: Model Training

5. Model Evaluation

Evaluating performance using Mean Absolute Error (MAE) and visualizing predictions vs actuals.

```
preds = rf_model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
rmse = np.sqrt(mean_squared_error(y_test, preds))

print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")

# Visualizing Prediction Accuracy
plt.figure(figsize=(10,6))
plt.scatter(y_test, preds, alpha=0.5, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Attendance')
plt.ylabel('Predicted Attendance')
plt.title('Prediction vs Actual Accuracy')
plt.show()
```

Figure 26: Model Evaluation

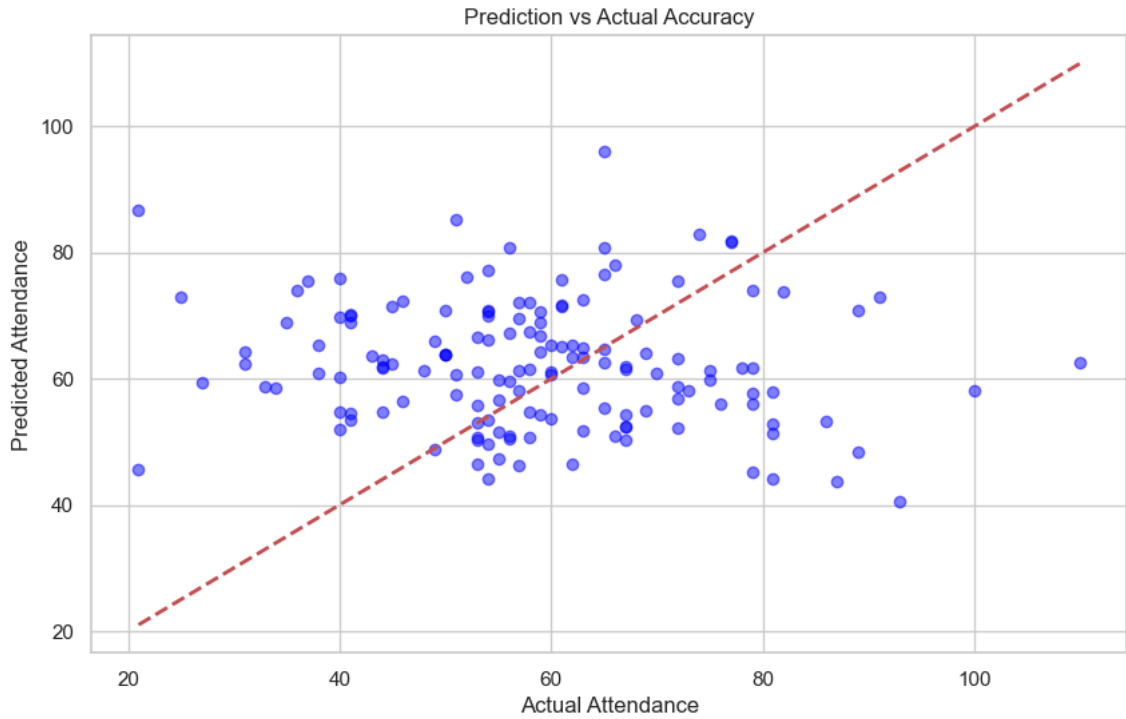


Figure 27: Model Evaluation

6. Saving the Model

Saving the trained model for backend integration.

```
import os
os.makedirs('./models', exist_ok=True)
joblib.dump(rf_model, './models/forecasting_model.pkl')
print("Forecast Model saved successfully to models/forecasting_model.pkl")
```

Figure 28: Saving the Model

3.2 Visitor Pattern Analysis and Segmentation

Perform comprehensive EDA on visitor demographics and behaviors, cluster visitors into segments (Unsupervised Learning), and build a classifier to predict event preferences (Supervised Learning).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, confusion_matrix
import joblib

# Configuration
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
```

Figure 29: Visitor Pattern Analysis and Segmentation

Data Loading

Loading the visitor logs dataset.

```
df = pd.read_csv('../visitor_data.csv')
df.head()
```

	visit_date	visitor_name	age	country	event_selected	tickets_count	total_spend_usd
0	2023-07-17	Visitor_40060	53	Sri Lanka	General Entry	2	2
1	2024-05-19	Visitor_87702	20	Sri Lanka	Bottle Feeding (Morning)	2	2
2	2023-03-11	Visitor_69730	40	Sri Lanka	General Entry	1	1
3	2023-03-10	Visitor_10046	61	Germany	Bottle Feeding (Afternoon)	3	45
4	2023-03-03	Visitor_19066	53	China	General Entry	3	45

Figure 30: Data Loading

Exploratory Data Analysis (EDA)

1. Visitor Demographics (Country & Age)

Understanding who visits the orphanage.

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# Country Distribution
country_counts = df['country'].value_counts()
sns.barplot(x=country_counts.values, y=country_counts.index, ax=axes[0], palette='coolwarm')
axes[0].set_title('Visitor Distribution by Country')

# Age Distribution
sns.histplot(df['age'], kde=True, bins=20, ax=axes[1], color='purple')
axes[1].set_title('Visitor Age Distribution')

plt.tight_layout()
plt.show()
```

Figure 31: Visitor Demographics

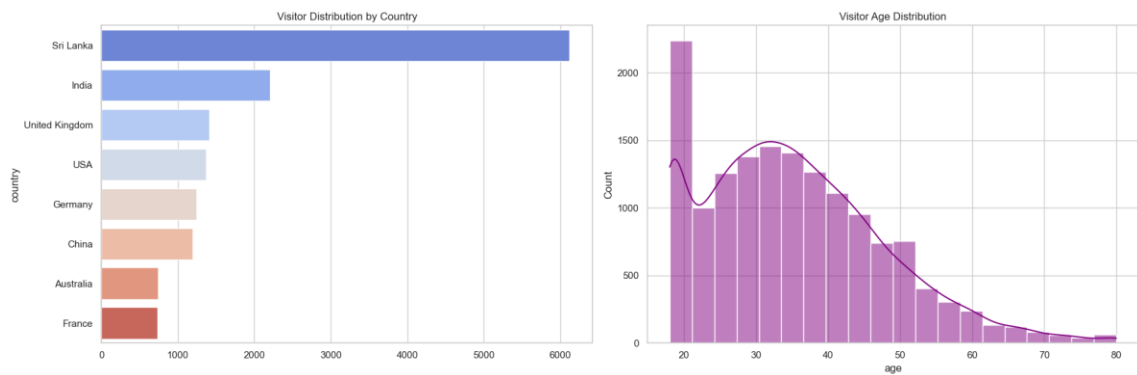


Figure 32: Visitor Demographics

2. Event Preferences by Demographics

Analyzing which events are popular among different countries.

```
plt.figure(figsize=(12,8))
top_countries = df['country'].value_counts().nlargest(5).index
filtered_df = df[df['country'].isin(top_countries)]

sns.countplot(data=filtered_df, y='event_selected', hue='country', palette='Set2')
plt.title('Event Preferences by Top 5 Visitor Nationalities')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.show()
```

Figure 33: Event Preferences by Demographics

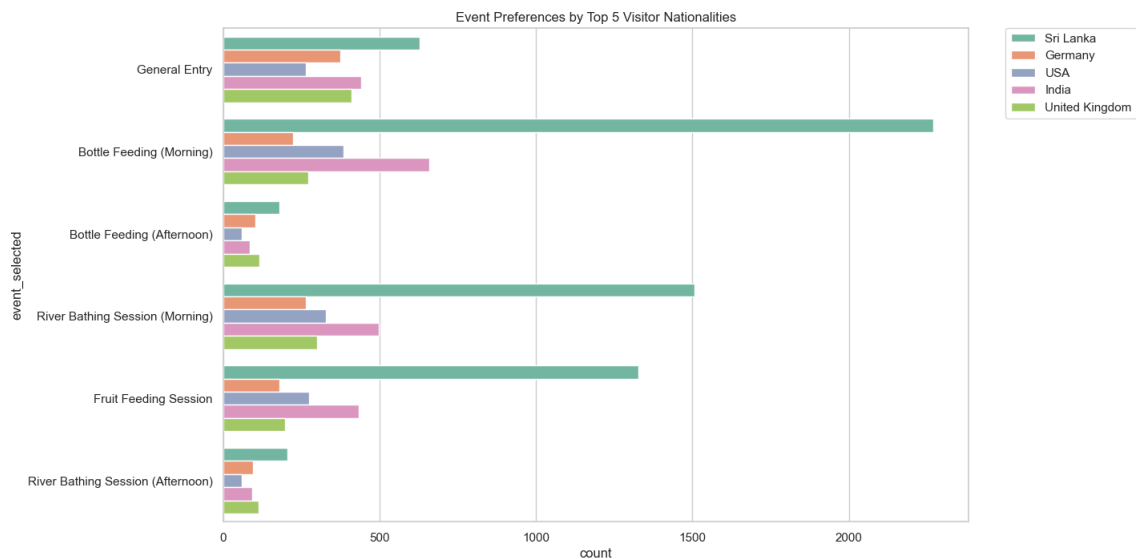


Figure 34: Event Preferences by Demographics

3 .Ticket Spending Analysis

Relationship between Age and Spending.

```
plt.figure(figsize=(10,6))
sns.scatterplot(data=df.sample(1000), x='age', y='total_spend_usd', hue='country', alpha=0.6)
plt.title('Age vs. Total Spending (Sampled 1000 records)')
plt.show()
```

Figure 35:Ticket Spending Analysis

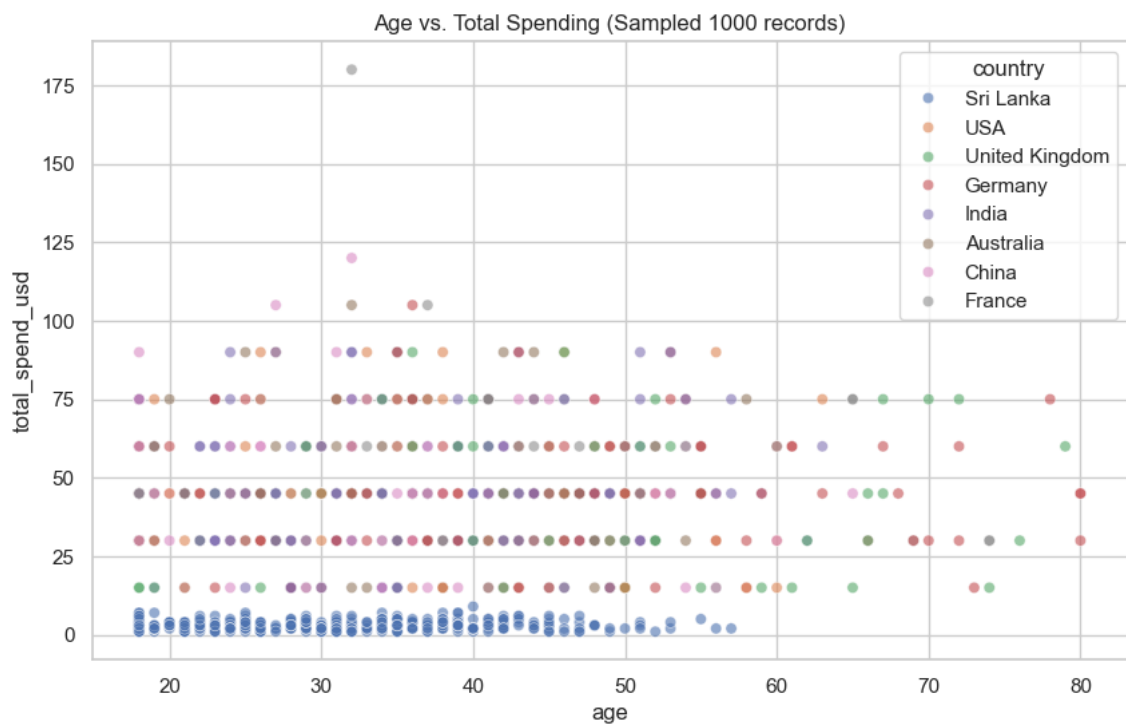


Figure 36:Ticket Spending Analysis

Visitor Segmentation (Clustering)

Using K-Means to identify distinct visitor segments based on Age, Country, and Ticket Counts.

```
# Preprocessing for Clustering
le_country = LabelEncoder()
df['country_code'] = le_country.fit_transform(df['country'])

features = df[['age', 'country_code', 'tickets_count']]

# K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df['cluster'] = kmeans.fit_predict(features)

# Visualizing Clusters
plt.figure(figsize=(10,6))
sns.scatterplot(data=df.sample(2000), x='age', y='tickets_count', hue='cluster', palette='deep', s=50)
plt.title('Visitor Segments: Age vs Tickets Purchased')
plt.show()
```

Figure 37: Visitor Segmentation

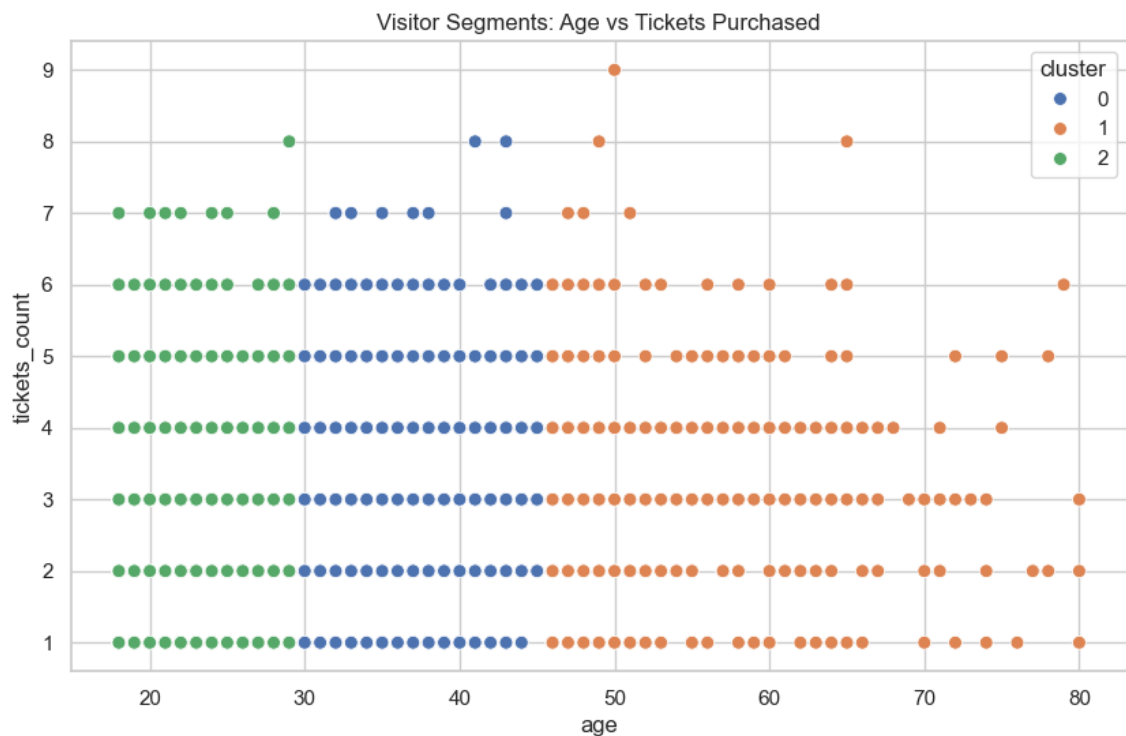


Figure 38: Visitor Segmentation

Preference Prediction (Classification)

Training a model to predict the 'Preferred Event' for a new visitor.

```
# Encoding Target Variable
le_event = LabelEncoder()
df['event_code'] = le_event.fit_transform(df['event_selected'])

X = df[['age', 'country_code']]
y = df['event_code']

# Train Model
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X, y)
print("Classification Model Trained.")
```

Figure 39: Preference Prediction

Saving Artifacts

Saving models and encoders for the live application.

```
joblib.dump(clf, '../models/preference_model.pkl')
joblib.dump(le_country, '../models/le_country.pkl')
joblib.dump(le_event, '../models/le_event.pkl')
joblib.dump(kmeans, '../models/clustering_model.pkl')
print("All models saved successfully.")
```

Figure 40: Saving Artifacts

3.3 Advanced Visitor Forecasting & EDA (V4)

Train models on "Yearly Waterfall" data (2022-2025) and forecast Jan 2026.

1. Separate EDA for each year.
2. Combined EDA.
3. Feature Engineering.
4. Model Training (Random Forest).
5. 14-Day Hourly Forecast.

```
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import LabelEncoder
import joblib
import warnings
warnings.filterwarnings('ignore')

# Config
DATA_DIR = '../datasets/advanced_hourly'
MODEL_DIR = '../models'
os.makedirs(MODEL_DIR, exist_ok=True)
```

Figure 41: Advanced Visitor Forecasting

Individual Year EDA

```
print("Loading 2022...")
try:
    df_2022 = pd.read_csv(f"{DATA_DIR}/visitor_dataset_2022.csv")
    df_2022['date'] = pd.to_datetime(df_2022['date'])
    print(f"2022: {len(df_2022)} rows")

    # Quick Plot
    daily_2022 = df_2022.groupby('date')['visitor_count'].sum()
    plt.figure(figsize=(10, 3))
    plt.plot(daily_2022, color='teal', alpha=0.6)
    plt.title(f"2022 Visitor Trend")
    plt.show()
except FileNotFoundError:
    print(f"Warning: 2022 file not found.")
    df_2022 = pd.DataFrame()
```

Figure 42: Individual Year EDA

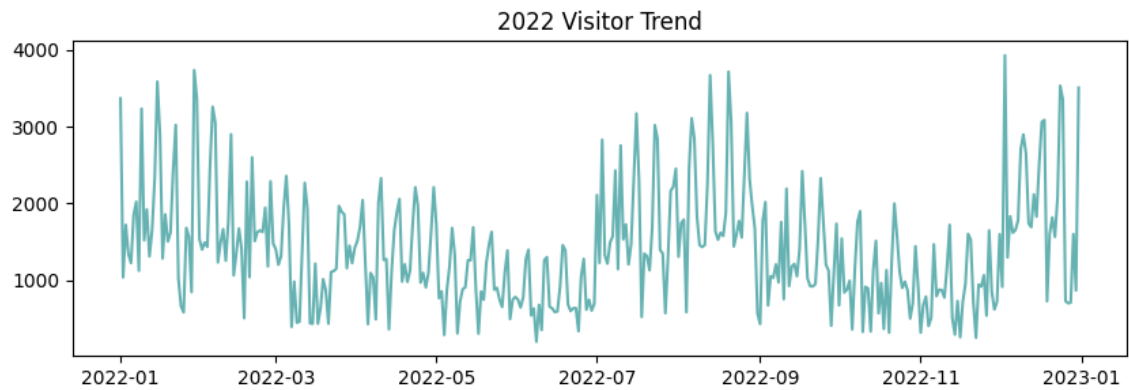


Figure 43: Individual Year EDA

```

print("Loading 2023...")
try:
    df_2023 = pd.read_csv(f"{DATA_DIR}/visitor_dataset_2023.csv")
    df_2023['date'] = pd.to_datetime(df_2023['date'])
    print(f"2023: {len(df_2023)} rows")

    # Quick Plot
    daily_2023 = df_2023.groupby('date')['visitor_count'].sum()
    plt.figure(figsize=(10, 3))
    plt.plot(daily_2023, color='teal', alpha=0.6)
    plt.title(f"2023 Visitor Trend")
    plt.show()
except FileNotFoundError:
    print(f"Warning: 2023 file not found.")
    df_2023 = pd.DataFrame()

```

Figure 44: Individual Year EDA

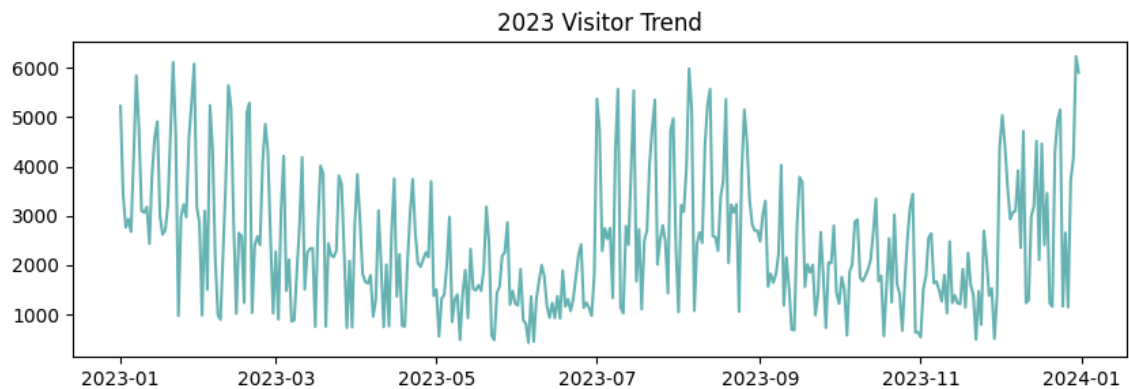


Figure 45: Individual Year EDA

```

print("Loading 2024...")
try:
    df_2024 = pd.read_csv(f"{DATA_DIR}/visitor_dataset_2024.csv")
    df_2024['date'] = pd.to_datetime(df_2024['date'])
    print(f"2024: {len(df_2024)} rows")

    # Quick Plot
    daily_2024 = df_2024.groupby('date')['visitor_count'].sum()
    plt.figure(figsize=(10, 3))
    plt.plot(daily_2024, color='teal', alpha=0.6)
    plt.title(f"2024 Visitor Trend")
    plt.show()
except FileNotFoundError:
    print(f"Warning: 2024 file not found.")
    df_2024 = pd.DataFrame()

```

Figure 46: Individual Year EDA

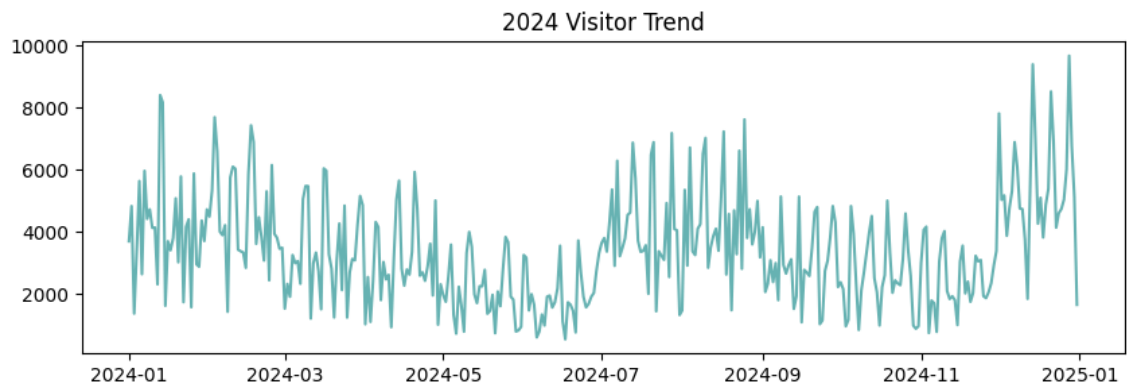


Figure 47: Individual Year EDA

```

print("Loading 2025...")
try:
    df_2025 = pd.read_csv(f"{DATA_DIR}/visitor_dataset_2025.csv")
    df_2025['date'] = pd.to_datetime(df_2025['date'])
    print(f"2025: {len(df_2025)} rows")

    # Quick Plot
    daily_2025 = df_2025.groupby('date')['visitor_count'].sum()
    plt.figure(figsize=(10, 3))
    plt.plot(daily_2025, color='teal', alpha=0.6)
    plt.title(f"2025 Visitor Trend")
    plt.show()
except FileNotFoundError:
    print(f"Warning: 2025 file not found.")
    df_2025 = pd.DataFrame()

```

Figure 48: Individual Year EDA

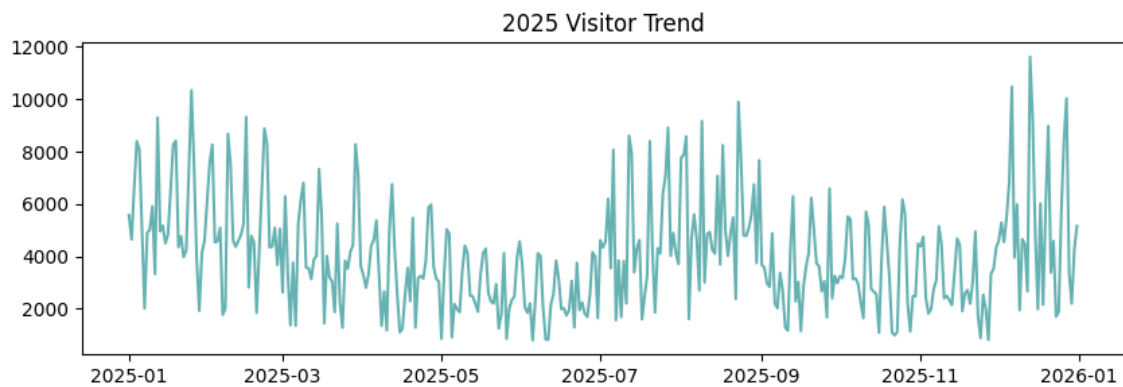


Figure 49: Individual Year EDA

Combined EDA & Feature Engineering

```
# Merge
full_df = pd.concat([df_2022, df_2023, df_2024, df_2025], ignore_index=True)
print(f"Full Dataset: {len(full_df)} rows")

# Features
full_df['date'] = pd.to_datetime(full_df['date'])
full_df['day_of_year'] = full_df['date'].dt.dayofyear
full_df['month'] = full_df['date'].dt.month
full_df['dow'] = full_df['date'].dt.dayofweek
full_df['is_weekend'] = (full_df['dow'] >= 5).astype(int)
full_df['hour_int'] = full_df['hour'].str.split(':').str[0].astype(int)

# Encoding (One-Hot for Location/Event to preserve nuances)
df_model = pd.get_dummies(full_df, columns=['location', 'event_name'], drop_first=False)

# Select Features
exclude = ['date', 'hour', 'visitor_count', 'weather'] # excluding weather for forecast simplicity
features = [c for c in df_model.columns if c not in exclude]

print(f"Training Features ({len(features)}): {features[:5]} ...")
X = df_model[features]
y = df_model['visitor_count']
```

Figure 50: Combined EDA & Feature Engineering

Model Training

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=100, max_depth=20, random_state=42)
model.fit(X_train, y_train)

preds = model.predict(X_test)
print(f"MAE: {mean_absolute_error(y_test, preds):.2f}")
print(f"R2: {r2_score(y_test, preds):.4f}")

# Save Model
joblib.dump(model, f"{MODEL_DIR}/admin_hourly_model.pkl")
# Save Feature Columns (Critical for prediction alignment)
joblib.dump(features, f"{MODEL_DIR}/model_features.pkl")
print("Model & Features Saved.")
```

Figure 51: Model Training

Next 14 Days Forecast

```
# Setup Future DataFrame
last_date = full_df['date'].max()
future_dates = [last_date + pd.Timedelta(days=i) for i in range(1, 15)]
hours = range(8, 18)
locations = full_df['location'].unique()

rows = []

# Helper for Event Logic (matches generator)
def get_event(loc, h):
    if loc == "Elephant Bathing Area":
        return "River Bathing" if h in [10, 11, 14, 15] else "River Observation"
    if loc == "Milk Feeding Nursery":
        return "Bottle Feeding" if h in [9, 13, 17] else "Closed"
    if loc == "Main Entrance":
        return "Entry"
    if loc == "Souvenir Shop":
        return "Shopping"
    return "General Visit"

print("Generating Future Inputs...")

for date in future_dates:
    for h in hours:
        for loc in locations:
            event = get_event(loc, h)

            # Base features
            row = {
                'day_of_year': date.dayofyear,
                'month': date.month,
                'dow': date.dayofweek,
                'is_weekend': int(date.dayofweek >= 5),
                'hour_int': h
```

Figure 52: Next 14 Days Forecast

```

# Base features
row = {
    'day_of_year': date.dayofyear,
    'month': date.month,
    'dow': date.dayofweek,
    'is_weekend': int(date.dayofweek >= 5),
    'hour_int': h
}

# One-Hot Columns (Set 1 or 0)
# Init all dummy columns to 0
for feat in features:
    if feat not in row:
        row[feat] = 0

# Set Active Dummies
if f"location_{loc}" in row: row[f"location_{loc}"] = 1
if f"event_name_{event}" in row: row[f"event_name_{event}"] = 1

# Metadata for result
row['date'] = date
row['hour'] = f"{h:02d}:00"
row['location'] = loc
row['event'] = event

rows.append(row)

pred_df = pd.DataFrame(rows)

# Predict
X_future = pred_df[features]
pred_df['predicted_visitors'] = model.predict(X_future).astype(int)

# Clean Output

```

Figure 53: Next 14 Days Forecast

```

# Clean Output
output = pred_df[['date', 'hour', 'location', 'event', 'predicted_visitors']]
print(output.head())

# Save
output.to_csv(f"{DATA_DIR}/forecast_14_days.csv", index=False)
print("Forecast Saved.")

```

Figure 54: Next 14 Days Forecast

```

Generating Future Inputs...
      date    hour      location      event \
0 2026-01-01 08:00      Main Entrance      Entry
1 2026-01-01 08:00  Elephant Bathing Area  River Observation
2 2026-01-01 08:00   Milk Feeding Nursery      Closed
3 2026-01-01 08:00           Museum      General Visit
4 2026-01-01 08:00      Souvenir Shop      Shopping

      predicted_visitors
0                      72
1                      35
2                      60
3                      41
4                      16
Forecast Saved.

```

Figure 55:Next 14 Days Forecast

4 CONCLUSION

This research successfully developed a mobile application-based Visitor Data Analytics and Experience Optimization system for the Pinnawala Elephant Orphanage. The study addressed the key problem of the lack of a data-driven approach to understanding visitor behavior and managing visitor flow. By replacing traditional manual methods with a digital solution, the system was able to collect, process, and analyze visitor data in a structured and efficient manner.

The results demonstrated that visitor patterns such as peak visiting times, popular activities, and visitor preferences can be effectively identified using data analytics and machine learning techniques. The implementation of the Random Forest Regressor model enabled accurate prediction of visitor trends, supporting proactive decision-making. Furthermore, the development of an interactive management dashboard provided clear visualization of insights, allowing staff to make informed decisions regarding event scheduling and crowd management.

The proposed solution significantly improves visitor experience by reducing overcrowding and enhancing the organization of activities. At the same time, it contributes to elephant welfare by helping to minimize stress caused by high visitor concentration. The use of a mobile application as the primary data collection method ensures that the system is cost-effective, scalable, and easy to implement without requiring complex infrastructure.

In conclusion, this research highlights the importance of integrating mobile technology, data analytics, and machine learning in managing high-traffic tourist destinations. The developed system provides a practical and sustainable approach to balancing tourism benefits with conservation needs. The solution can be further extended and adapted to other wildlife parks and tourist attractions, contributing to the advancement of smart and ethical tourism practices.

5 REFERENCES

Althoff, T., Sasic, R., Hicks, J. L., King, A. C., Delp, S. L., & Leskovec, J. (2017). Large-scale physical activity data reveal worldwide activity inequality. Nature, 547(7663), 336-339.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Buhalis, D., & Amaranggana, A. (2015). Smart tourism destinations. In Tourism in the digital age (pp. 55-81). Springer.

Ceron-Carrasco, J. P. (2022). Deep learning for wildlife conservation: A review of current status and future prospects. Ecological Informatics, 70, 101732.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.

Clubb, R., & Mason, G. J. (2002). A review of the welfare of zoo elephants in Europe. RSPCA.

Fernando, P., & Pastorini, J. (2011). Range use and human-elephant conflict in Sri Lanka. Gajah, 34, 4-6.

Hampton, J. O., Hyndman, T. H., Barnes, A., & Collins, T. (2020). Is wildlife management humane? Journal of Applied Animal Welfare Science, 23(3), 263-278.

ISO/IEC 25010:2011. (2011). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. International Organization for Standardization.

Kandhari, A., & Sharma, V. (2020). Visitor management and crowd control in wildlife sanctuaries: A data-driven approach. International Journal of Tourism Research, 22(4), 488-500.

Kruse, R., Mostaghim, S., Borgelt, C., Braune, C., & Steinbrecher, M. (2022). Computational Intelligence: A Methodological Introduction. Springer.

Lofgren, O. (2019). The Tourist Gaze and the History of Sightseeing. University of California Press.

Mason, G. J., & Veasey, J. S. (2010). What do population-level welfare indices suggest about the welfare of captive elephants? Zoo Biology, 29(2), 256-273.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Ponnambalam, S. G., & Kanagaraj, G. (2021). Visitor crowd management using IoT and machine learning in smart tourism. Journal of Ambient Intelligence and Humanized Computing, 12, 5671-5685.

Santiapillai, C., & Wijeyamohan, S. (2003). The elephant transit home at Udawalawe. Gajah, 22, 60-63.

Zhang, J., & Wang, Y. (2020). Big data analytics in smart tourism: A comprehensive review. Tourism Management Perspectives, 35, 100683.